

Ejercicios

ACLARACIÓN Los ejercicios están resueltos, pero no está explicado el porqué de cada caso como hicimos en clase 😊

1. Ejercicio 1: Contador de visitas

Supongamos que queremos llevar un registro de las visitas a un sitio web. Podemos utilizar una lista enlazada para almacenar la información de cada visita. Cada nodo de la lista podría contener los datos de la visita, como la fecha, la hora y la dirección IP del visitante. Para agregar una nueva visita, simplemente insertamos un nuevo nodo al final de la lista. Para contar el número total de visitas, recorremos la lista y contamos la cantidad de nodos.

```
class Nodo:
    def __init__(self, fecha, hora, ip):
        self.fecha = fecha
        self.hora = hora
        self.ip = ip
        self.siguiente = None

class ListaEnlazada:
    def __init__(self):
        self.cabeza = None

    def agregar_visita(self, fecha, hora, ip):
        nuevo_nodo = Nodo(fecha, hora, ip)
        if not self.cabeza:
            self.cabeza = nuevo_nodo
        else:
            actual = self.cabeza
            while actual.siguiente:
                actual = actual.siguiente
            actual.siguiente = nuevo_nodo

    def contar_visitas(self):
        contador = 0
        actual = self.cabeza
        while actual:
            contador += 1
            actual = actual.siguiente
        return contador
```

2. Ejercicio 2: Lista de tareas pendientes

Imaginemos que queremos crear una aplicación para gestionar nuestras tareas pendientes. Podemos utilizar una lista enlazada para almacenar las tareas. Cada nodo de la lista podría contener los datos de una tarea, como el título, la descripción y la fecha límite. Para agregar una nueva tarea,

insertamos un nuevo nodo al final de la lista. Para marcar una tarea como completada, podemos eliminar el nodo correspondiente de la lista. Para mostrar todas las tareas pendientes, recorremos la lista y mostramos los datos de cada nodo.

```
class Nodo:
    def __init__(self, titulo, descripcion, fecha_limite):
        self.titulo = titulo
        self.descripcion = descripcion
        self.fecha_limite = fecha_limite
        self.siguiente = None

class PilaDeTareas:
    def __init__(self):
        self.cima = None

    def agregar_tarea(self, titulo, descripcion, fecha_limite):
        nuevo_nodo = Nodo(titulo, descripcion, fecha_limite)
        nuevo_nodo.siguiente = self.cima
        self.cima = nuevo_nodo

    def eliminar_tarea(self):
        if self.cima:
            self.cima = self.cima.siguiente

    def mostrar_tareas(self):
        actual = self.cima
        while actual:
            print(f"Título: {actual.titulo}, Descripción: {actual.descripcion},  
Fecha Límite: {actual.fecha_limite}")
            actual = actual.siguiente
```

3. Ejercicio 3: Historial de reproducción de música

Supongamos que queremos llevar un registro de las canciones que hemos reproducido en un reproductor de música. Podemos utilizar una lista enlazada para almacenar la información de cada canción. Cada nodo de la lista podría contener los datos de la canción, como el título, el artista y la duración. Para agregar una nueva canción al historial, insertamos un nuevo nodo al inicio de la lista. De esta manera, siempre tendremos la canción más reciente al principio. Para mostrar el historial de reproducción, recorremos la lista enlazada y mostramos los datos de cada nodo.

```
class Nodo:
    def __init__(self, titulo, artista, duracion):
        self.titulo = titulo
        self.artista = artista
        self.duracion = duracion
        self.siguiente = None
```

```

class HistorialReproduccion:
    def __init__(self):
        self.cabeza = None
        self cola = None

    def agregar_cancion(self, titulo, artista, duracion):
        nuevo_nodo = Nodo(titulo, artista, duracion)
        if not self.cabeza:
            self.cabeza = nuevo_nodo
            self.col a = nuevo_nodo
        else:
            self.col a.sigui ente = nuevo_nodo
            self.col a = nuevo_nodo

    def mostrar_historial(self):
        actual = self.cabeza
        while actual:
            print(f"Título: {actual.titulo}, Artista: {actual.artista},
Duración: {actual.duracion}")
            actual = actual.sigui ente

```

4. Ejercicio 4: Registro de ventas

Imaginemos que queremos llevar un registro de las ventas realizadas en una tienda. Podemos utilizar una lista enlazada para almacenar la información de cada venta. Cada nodo de la lista podría contener los datos de la venta, como el nombre del cliente, los productos comprados y el monto total. Para agregar una nueva venta, insertamos un nuevo nodo al final de la lista. Para calcular el total de ventas realizadas, recorremos la lista y sumamos los montos de cada venta.

```

class Nodo:
    def __init__(self, cliente, productos, monto):
        self.cliente = cliente
        self.productos = productos
        self.monto = monto
        self.sigui ente = None

class RegistroVentas:
    def __init__(self):
        self.cabeza = None

    def agregar_venta(self, cliente, productos, monto):
        nuevo_nodo = Nodo(cliente, productos, monto)
        if not self.cabeza:
            self.cabeza = nuevo_nodo
        else:
            actual = self.cabeza
            while actual.sigui ente:
                actual = actual.sigui ente
            actual.sigui ente = nuevo_nodo

```

```

def total_ventas(self):
    total = 0
    actual = self.cabeza
    while actual:
        total += actual.monto
        actual = actual.siguiente
    return total

def mostrar_ventas(self):
    actual = self.cabeza
    while actual:
        print(f"Cliente: {actual.cliente}, Productos: {actual.productos},
Monto: {actual.monto}")
        actual = actual.siguiente

```

5. Ejercicio 5: Gestión de contactos

Supongamos que queremos crear una aplicación para gestionar nuestros contactos. Podemos utilizar una lista enlazada para almacenar la información de cada contacto. Cada nodo de la lista podría contener los datos del contacto, como el nombre, el número de teléfono y el correo electrónico. Para agregar un nuevo contacto, insertamos un nuevo nodo al final de la lista. Para buscar un contacto por nombre, recorremos la lista y comparamos el nombre de cada nodo con el nombre buscado. Si encontramos una coincidencia, mostramos los datos del contacto. Si queremos eliminar un contacto, podemos eliminar el nodo correspondiente de la lista.

```

class Nodo:
    def __init__(self, nombre, telefono, correo):
        self.nombre = nombre
        self.telefono = telefono
        self.correo = correo
        self.siguiente = None
        self.anterior = None

class ListaContactos:
    def __init__(self):
        self.cabeza = None
        self cola = None

    def agregar_contacto(self, nombre, telefono, correo):
        nuevo_nodo = Nodo(nombre, telefono, correo)
        if not self.cabeza:
            self.cabeza = nuevo_nodo
            self.cola = nuevo_nodo
        else:
            self.cola.siguiente = nuevo_nodo
            nuevo_nodo.anterior = self.cola
            self.cola = nuevo_nodo

```

```

def buscar_contacto(self, nombre):
    actual = self.cabeza
    while actual:
        if actual.nombre == nombre:
            print(f"Nombre: {actual.nombre}, Teléfono: {actual.telefono},
Correo: {actual.correo}")
            return
        actual = actual.siguiente
    print("Contacto no encontrado")

def eliminar_contacto(self, nombre):
    actual = self.cabeza
    while actual:
        if actual.nombre == nombre:
            if actual.anterior:
                actual.anterior.siguiente = actual.siguiente
            if actual.siguiente:
                actual.siguiente.anterior = actual.anterior
            if actual == self.cabeza:
                self.cabeza = actual.siguiente
            if actual == self.colta:
                self.colta = actual.anterior
            return
        actual = actual.siguiente
    print("Contacto no encontrado")

def mostrar_contactos(self):
    actual = self.cabeza
    while actual:
        print(f"Nombre: {actual.nombre}, Teléfono: {actual.telefono},
Correo: {actual.correo}")
        actual = actual.siguiente

```

Para los cinco ejercicios anteriores, pensar si sería mejor una lista enlazada, una pila o una cola

6. Ejercicio 6: Organización de archivos

Imaginemos que queremos organizar nuestros archivos en una estructura de árbol. Podemos utilizar un árbol binario para representar la jerarquía de carpetas y archivos. Cada nodo del árbol podría contener los datos de una carpeta o archivo, como el nombre y el tamaño. Para organizar los archivos, insertamos nuevos nodos en el árbol de acuerdo a su ubicación en la estructura de carpetas. Para buscar un archivo, recorremos el árbol y comparamos el nombre de cada nodo con el nombre buscado. Si encontramos una coincidencia, mostramos los datos del archivo.

```

class Nodo:
    def __init__(self, nombre, tamano):
        self.nombre = nombre

```

```

        self.tamano = tamano
        self.hijos = []

class ArbolArchivos:
    def __init__(self):
        self.raiz = None

    def agregar_archivo(self, ruta, nombre, tamano):
        if not self.raiz:
            self.raiz = Nodo(nombre, tamano)
        else:
            nodos = ruta.split('/')
            actual = self.raiz
            for nodo in nodos:
                encontrado = False
                for hijo in actual.hijos:
                    if hijo.nombre == nodo:
                        actual = hijo
                        encontrado = True
                        break
                if not encontrado:
                    nuevo_nodo = Nodo(nodo, 0)
                    actual.hijos.append(nuevo_nodo)
                    actual = nuevo_nodo
            actual.hijos.append(Nodo(nombre, tamano))

    def buscar_archivo(self, nombre):
        return self._buscar_archivo_recursivo(self.raiz, nombre)

    def _buscar_archivo_recursivo(self, nodo, nombre):
        if nodo.nombre == nombre:
            return nodo
        for hijo in nodo.hijos:
            resultado = self._buscar_archivo_recursivo(hijo, nombre)
            if resultado:
                return resultado
        return None

    def mostrar_arbol(self):
        self._mostrar_arbol_recursivo(self.raiz, 0)

    def _mostrar_arbol_recursivo(self, nodo, nivel):
        print('  ' * nivel + f"{nodo.nombre} ({nodo.tamano} bytes)")
        for hijo in nodo.hijos:
            self._mostrar_arbol_recursivo(hijo, nivel + 1)

```

7. Ejercicio 7: Árbol genealógico

Supongamos que queremos representar un árbol genealógico de una familia. Podemos utilizar un árbol general para representar las relaciones entre los miembros de la familia. Cada nodo del árbol

podría contener los datos de una persona, como el nombre, la fecha de nacimiento y el parentesco. Para construir el árbol genealógico, insertamos nuevos nodos en el árbol de acuerdo a las relaciones familiares. Para buscar un miembro de la familia, recorremos el árbol y comparamos el nombre de cada nodo con el nombre buscado. Si encontramos una coincidencia, mostramos los datos de la persona.

Como Árbol:

```
class Nodo:
    def __init__(self, nombre, fecha_nacimiento, parentesco):
        self.nombre = nombre
        self.fecha_nacimiento = fecha_nacimiento
        self.parentesco = parentesco
        self.hijos = []

class ArbolGenealogico:
    def __init__(self):
        self.raiz = None

    def agregar_miembro(self, ruta, nombre, fecha_nacimiento, parentesco):
        if not self.raiz:
            self.raiz = Nodo(nombre, fecha_nacimiento, parentesco)
        else:
            nodos = ruta.split('/')
            actual = self.raiz
            for nodo in nodos:
                encontrado = False
                for hijo in actual.hijos:
                    if hijo.nombre == nodo:
                        actual = hijo
                        encontrado = True
                        break
                if not encontrado:
                    nuevo_nodo = Nodo(nodo, "", "")
                    actual.hijos.append(nuevo_nodo)
                    actual = nuevo_nodo
            actual.hijos.append(Nodo(nombre, fecha_nacimiento, parentesco))

    def buscar_miembro(self, nombre):
        return self._buscar_miembro_recursivo(self.raiz, nombre)

    def _buscar_miembro_recursivo(self, nodo, nombre):
        if nodo.nombre == nombre:
            return nodo
        for hijo in nodo.hijos:
            resultado = self._buscar_miembro_recursivo(hijo, nombre)
            if resultado:
                return resultado
        return None

    def mostrar_arbol(self):
```

```

        self._mostrar_arbol_recursivo(self.raiz, 0)

    def _mostrar_arbol_recursivo(self, nodo, nivel):
        print('  ' * nivel + f"{nodo.nombre} ({nodo.fecha_nacimiento}, {nodo.parentesco})")
        for hijo in nodo.hijos:
            self._mostrar_arbol_recursivo(hijo, nivel + 1)

```

Como Grafo:

```

class Nodo:
    def __init__(self, nombre, fecha_nacimiento, parentesco):
        self.nombre = nombre
        self.fecha_nacimiento = fecha_nacimiento
        self.parentesco = parentesco
        self.relaciones = []

class GrafoGenealogico:
    def __init__(self):
        self.nodos = {}

    def agregar_miembro(self, nombre, fecha_nacimiento, parentesco):
        if nombre not in self.nodos:
            self.nodos[nombre] = Nodo(nombre, fecha_nacimiento, parentesco)

    def agregar_relacion(self, nombre1, nombre2, tipo_relacion):
        if nombre1 in self.nodos and nombre2 in self.nodos:
            self.nodos[nombre1].relaciones.append((self.nodos[nombre2],
            tipo_relacion))
            self.nodos[nombre2].relaciones.append((self.nodos[nombre1],
            tipo_relacion))

    def buscar_miembro(self, nombre):
        return self.nodos.get(nombre, None)

    def mostrar_grafo(self):
        for nombre, nodo in self.nodos.items():
            print(f"{nombre} ({nodo.fecha_nacimiento}, {nodo.parentesco})")
            for relacion, tipo in nodo.relaciones:
                print(f"  -> {relacion.nombre} ({tipo})")

```